Flight Delay Prediction

Predicting US domestic arrival delays from a distributed Big Data pipeline

Team 22

Artem Panov · Arthur Babkin · Anastasiia Ozerova · Alexander Malyy

Introduction to Big Data, Innopolis University, May 2026

THE PROBLEM

> *Can we predict, before pushback, whether a US domestic flight will arrive* ***more than 15 minutes late****?*

~20 %
of US flights delayed (BTS 2024)

$30 B+
annual cost to US aviation

7 M
flight records / year

4 : 1
on-time vs delayed imbalance

Stakeholders

Airlines
Gate, crew, late-aircraft penalties

Airports
Turnaround scheduling

Passengers
Missed connections

OUR GOAL

01
Identify the patterns

Quantify how arrival delay varies across carriers, routes, airports, departure hours, and seasons. Ten EDA queries on 7 M flights.

02
Predict before pushback

Build a binary classifier for `arr_delay > 15` using only pre-departure features: scheduled times, route, carrier, distance, calendar.

03
Run it at scale

End-to-end on a distributed Hadoop stack (Postgres → Sqoop → HDFS → Hive → Spark ML → Superset) so the same pipeline scales to multi-year flight data.

DATASET CHARACTERISTICS

Source. Kaggle `hrishitpatil/flight-data-2024`, derived from BTS On-Time Performance database.

7.08 M
flight records · 2024

36 → 40
cols (raw → engineered)

1.31 GB → 250 MB
CSV → Parquet+Snappy (5×)

12 × 16
month partitions × carrier buckets

Target: `is_arrival_delayed`

`= 1 if arr_delay > 15 else 0`

label	count	share
0 on-time	5,561,879	**79.85 %**
1 delayed	1,403,388	**20.15 %**

4 : 1 imbalance, handled with inverse-frequency class weights in Stage 3.

Cancelled + diverted (113,814 rows) excluded from training.

Sample flight record

```
flight_id          : 1
month              : 1
day_of_week        : 2
op_unique_carrier  : DL
origin / dest      : ATL / MIA
crs_dep_time       : 0805
crs_arr_time       : 1015
distance           : 594
arr_delay          : -3
cancelled          : 0
scheduled_dep_hour : 8
is_weekend         : 0
is_arrival_delayed : 0
```

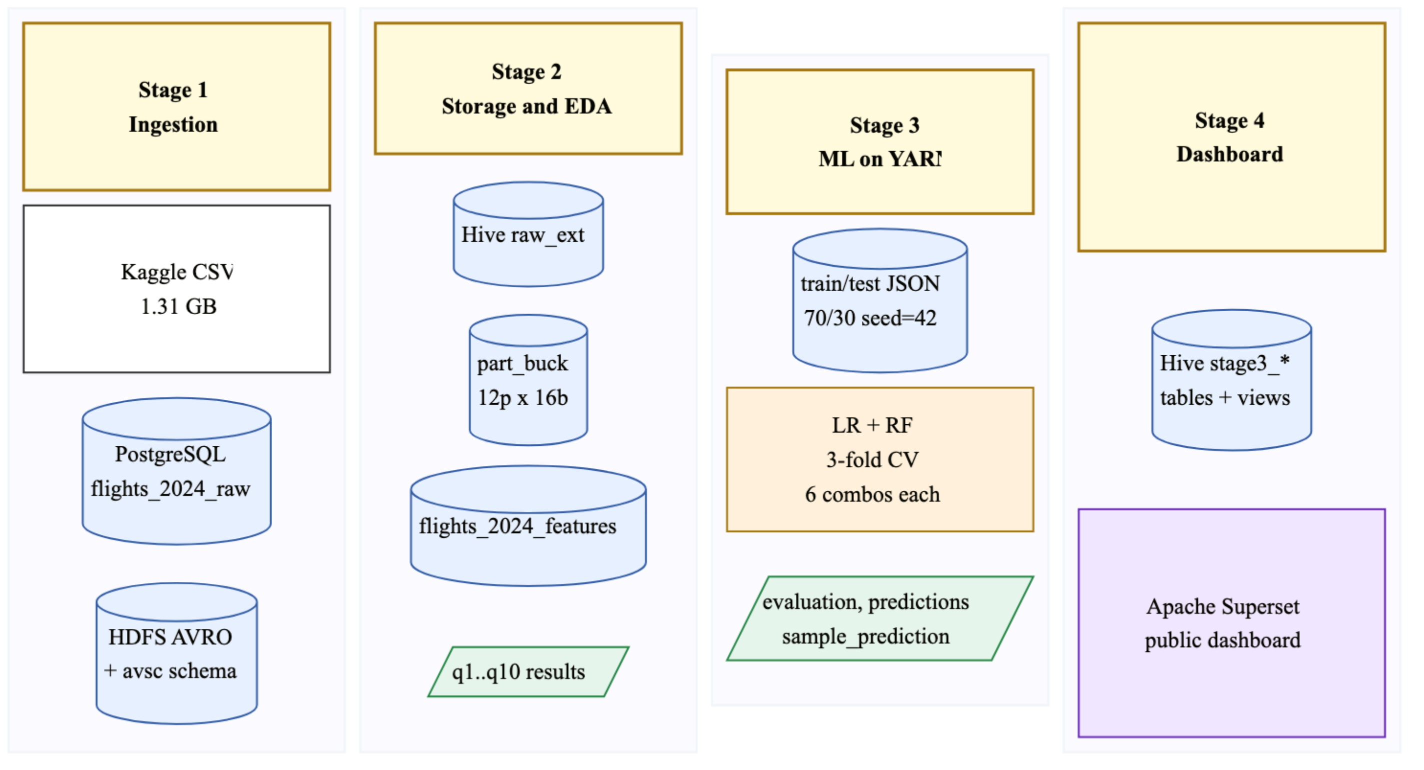
PIPELINE ARCHITECTURE

01 **Ingestion.** Postgres + Sqoop → AVRO/Snappy on HDFS

02 **Storage + EDA.** Hive Parquet, partitioned/bucketed; 10 EDA queries

03 **ML.** Spark ML on YARN, LR + RF with 3-fold CV

04 **Dashboard.** Hive views + Apache Superset



End-to-end via `bash main.sh`. Idempotent: every stage drops and rebuilds outputs.

01 STAGE 1: INGESTION

Postgres entry point. Single fact table `flights_2024_raw`, 36 cols mirroring BTS file. No star schema (analytical layer lives in Hive, not OLAP on Postgres).

Kaggle CSV 1.31 GB → **PostgreSQL** psycopg2 COPY → **HDFS AVRO** Sqoop + Snappy

Sqoop writes AVSC schema to `project/warehouse/avsc/` for the Hive layer. ER diagram on the right shows full Postgres schema (36 cols, no FKs).

flights_2024_raw

PK	flight_id	BIGSERIAL	wheels_off	NUMERIC(10,1)
	year	INTEGER	wheels_on	NUMERIC(10,1)
	month	SMALLINT	taxi_in	NUMERIC(10,2)
	day_of_month	SMALLINT	crs_arr_time	INTEGER
	day_of_week	SMALLINT	arr_time	NUMERIC(10,1)
	fl_date	DATE	arr_delay	NUMERIC(10,2)
	op_unique_carrier	VARCHAR(10)	cancelled	SMALLINT
	op_carrier_fl_num	NUMERIC(10,1)	cancellation_code	VARCHAR(1)
	origin	VARCHAR(10)	diverted	SMALLINT
	origin_city_name	VARCHAR(128)	crs_elapsed_time	NUMERIC(10,2)
	origin_state_nm	VARCHAR(128)	actual_elapsed_time	NUMERIC(10,2)
	dest	VARCHAR(10)	air_time	NUMERIC(10,2)
	dest_city_name	VARCHAR(128)	distance	NUMERIC(10,2)
	dest_state_nm	VARCHAR(128)	carrier_delay	INTEGER
	crs_dep_time	INTEGER	weather_delay	INTEGER
	dep_time	NUMERIC(10,1)	nas_delay	INTEGER
	dep_delay	NUMERIC(10,2)	security_delay	INTEGER
	taxi_out	NUMERIC(10,2)	late_aircraft_delay	INTEGER

02 STAGE 2: STORAGE + EDA

Three Hive tables in sequence

1. `flights_2024_raw_ext`: external over AVRO
2. `flights_2024_part_buck`: Parquet+Snappy
3. `flights_2024_features`: final, +4 engineered

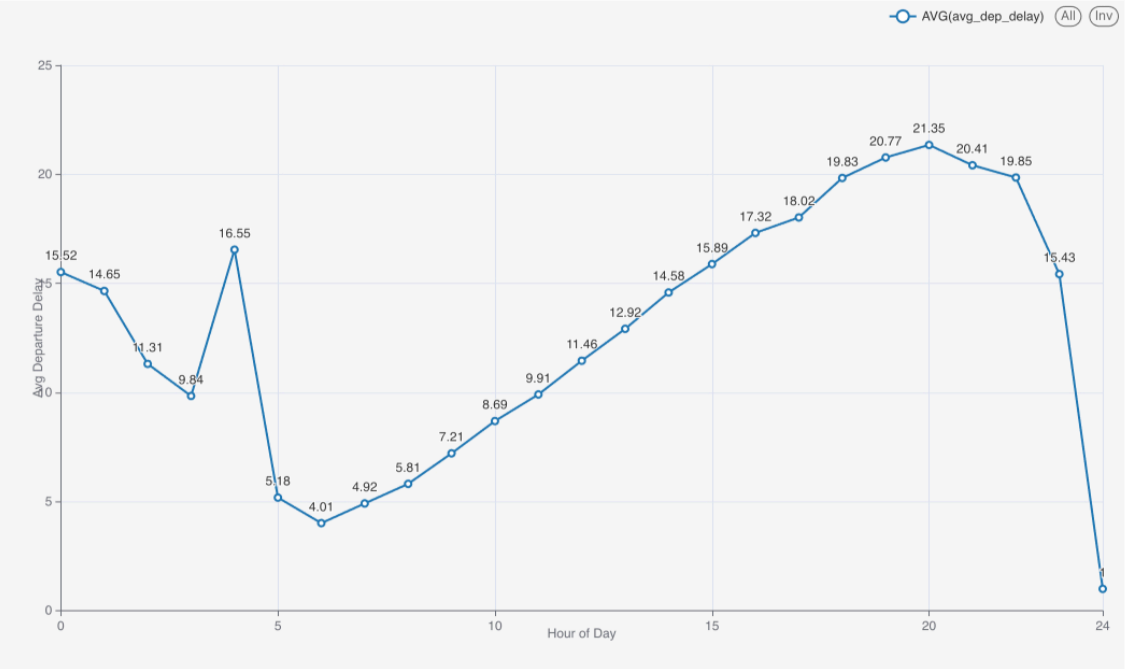
Layout

Partitioned by `month` (12). Natural seasonal slice.

Jan	Feb	Mar	Apr	May	Jun
Jul	Aug	Sep	Oct	Nov	Dec

Bucketed by `op_unique_carrier` (16). Even bucket sizes, fast joins.

10 HiveQL queries `q1.hql` .. `q10.hql` → CSV + Superset chart each. Feed Stage 4.



Teaser. q7: avg dep delay by hour. 5 to 7 AM lowest (~4 min), 8 PM peak (~20 min). We add cyclical sin/cos hour encoding to capture this curve. Full EDA in live demo.

03 STAGE 3: ML PIPELINE

StringIndexer × 3 (carrier · origin · dest) → **OneHotEncoder** → **VectorAssembler** + cyclical sin/cos + numeric → **StandardScaler** withMean=False → **LR / RF**

Engineered features

- `scheduled_dep_hour`, `scheduled_arr_hour`, `is_weekend`, target
- **Cyclical sin/cos** for hour, DOW, month, day-of-month. *Linear models can't see hour 23 ≈ hour 0 from a raw int.*

Imbalance & training

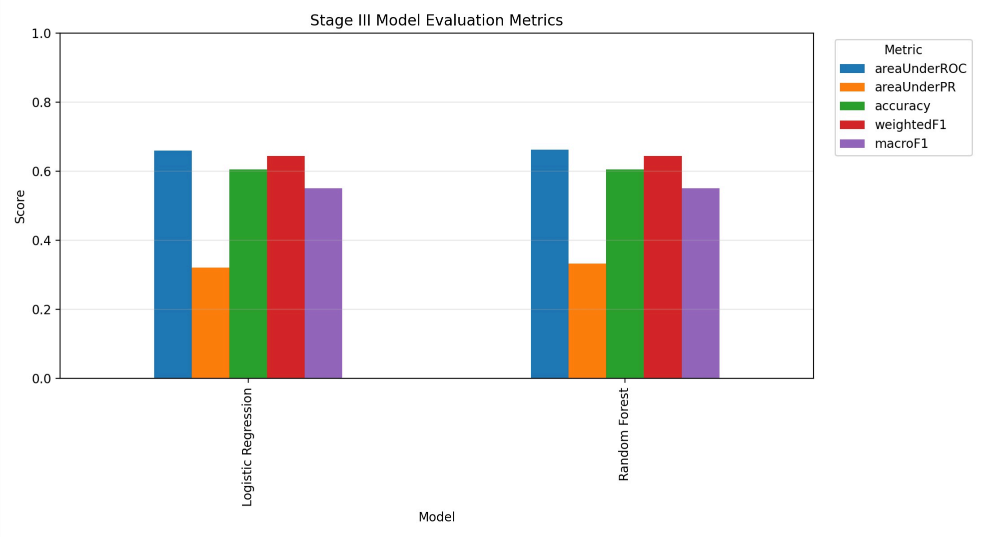
- Class weights from train set: $w_0 \approx 0.626$, $w_1 \approx 2.480$
- 70/30 split, seed 42 → 4.88 M train / 2.09 M test
- 3-fold CV, 6-combo grid per model

03

STAGE 3: RESULTS

Model	AUC-ROC	AUC-PR	Acc	F1
Logistic Regression	0.6602	0.3217	0.6048	0.6439
Random Forest (best)	**0.6622**	**0.3327**	0.6058	0.6446

- **Random Forest** wins all four metrics by a small margin
- AUC-PR **0.33** vs 0.20 baseline → lift on the rare (delayed) class
- **Ceiling.** With only pre-departure features we cap at ~0.66 ROC. The strongest signals (en-route weather, late-aircraft cascades) arrive after pushback, so we can't use them.



Sample prediction → `output/sample_prediction.csv` (live in dashboard).

04

LIVE DEMO

APACHE SUPERSET DASHBOARD

`hadoop-03.uni.innopolis.ru:8808/superset/dashboard/225/`

Three sections: Data Overview · Exploratory Analysis (10 charts) · Machine Learning

CHALLENGES WE HIT

Artem · Stage 1 plumbing

Friction with bash scripting and the Postgres schema setup. Getting `psycopg2` COPY and `sql/create_tables.sql` to run cleanly end-to-end took several iterations.

Arthur · Superset dashboard

Hardest part of Stage 4: chart layout, dataset registration. Also coordination work, since the Stage 2 feature table and the Stage 4 dashboard had to line up with what Stage 3 produced.

Anastasiia · ML on imbalanced data

Hyperparameter selection on the 4 : 1 split was hard. Sensible ranges for `regParam`, `numTrees`, `maxDepth` without overfitting needed several CV runs. Cluster issues and long Spark-queue waits (other teams hitting Stage 3 at the same time) slowed iteration.

Alexander · Team coordination

Trouble with team organization and communication led to some misunderstandings around task delegation.

FUTURE WORK + THANKS

Future work

- **Weather features.** q4 shows weather drives 56 % of cancellations.
- **Gradient-Boosted Trees** as a third model.
- **Target encoding** for `origin` × `dest` (replaces OHE on high-cardinality fields).
- **Streaming.** Kafka + Spark Structured Streaming for same-day forecasts.

Questions?

Repository

`github.com/n4rly-boop/ItoBD-project`

Report

`report/report.pdf`

Dashboard

`hadoop-03.uni.innopolis.ru:8808/superset/dashboard/225/`

Artem Panov · Arthur Babkin

Anastasiia Ozerova · Alexander Malyy